

Creating Calculated Fields

In this chapter, you will learn what calculated fields are, how to create them, and how to use aliases to refer to them from within your application.

Understanding Calculated Fields

Data stored within a database's tables is often not available in the exact format needed by your applications. Here are some examples:

- You need to display a field containing the name of a company along with the company's location, but that information is stored in separated table columns.
- City, state, and zip code are stored in separate columns (as they should be), but your mailing label printing program needs them retrieved as one correctly formatted field.
- Column data is in mixed upper- and lowercase, and your report needs data presented in all uppercase.
- An order items table stores item price and quantity but not the expanded price (that is price multiplied by quantity) for each item. To print invoices, you need that expanded price.
- You need totals, averages, or other calculations based on table data.

In each of these examples, the data stored in the table is not exactly what your application needs. Rather than retrieve the data as it is and then reformat it within your client application or report, you want to be able to retrieve converted, calculated, or reformatted data directly from the database.

This is where calculated fields come in. Unlike all the columns retrieved in the chapters thus far, calculated fields don't actually exist in database tables. Rather, a calculated field is created on-the-fly within a SQL `SELECT` statement.

New Term

Field Essentially the same thing as *column*. These terms are often used interchangeably, although database columns are typically called *columns* and the term *fields* is normally used in conjunction with calculated fields.

It is important to note that only the database knows which columns in a `SELECT` statement are actual table columns and which are calculated fields. From the perspective of a client (for example, your application), a calculated field's data is returned in the same way as data from any other column.

Tip

Client Versus Server Formatting Many of the conversions and reformatting operations that can be performed within SQL statements can also be performed directly in a client application. However, as a rule, it is far quicker to perform these operations on a database server than it is to perform them within a client because database management systems (DBMSs) are built to perform this type of processing quickly and efficiently.

Concatenating Fields

To demonstrate working with calculated fields, let's start with a simple example: creating a title made up of two columns.

The `vendors` table contains vendor name and address information. Imagine that you are generating a vendor report and need to list the vendor location as part of the vendor name in the format *name (location)*.

The report wants a single value, and the data in the table is stored in two columns: `vend_name` and `vend_country`. In addition, you need to surround `vend_country` with parentheses, and those are definitely not stored in the database table. The `SELECT` statement that returns the vendor names and locations is simple enough, but how would you create the combined value that you need?

The solution is to concatenate the two columns. In MySQL `SELECT` statements, you can concatenate columns by using the `Concat()` function.

New Term

Concatenate To join values together (by appending them to each other) to form a single long value.

Tip

MySQL Is Different Most DBMSs use the operators `+` or `||` for concatenation; MySQL uses the `Concat()` function. Keep this in mind when converting SQL statements to MySQL.

Here's an example of using the `Concat()` function:

► Input

```
SELECT Concat(vend_name, ' (', vend_country, ')')
FROM vendors
ORDER BY vend_name;
```

► Output

```
+-----+
| Concat(vend_name, ' (', vend_country, ')') |
+-----+
| ACME (USA)                                |
| Anvils R Us (USA)                        |
| Furball Inc. (USA)                       |
| Jet Set (England)                        |
| Jouets Et Ours (France)                  |
| LT Supplies (USA)                       |
+-----+
```

► Analysis

`Concat()` concatenates strings, appending them to each other to create one bigger string. `Concat()` requires one or more values to be specified, and the values need to be separated by commas. The `SELECT` statements in this example concatenate four elements:

- The name stored in the `vend_name` column
- A string containing a space and an open parenthesis
- The state stored in the `vend_country` column
- A string containing the close parenthesis

As you can see in the output, the `SELECT` statement returns a single column (a calculated field) that contains all four of these elements as one unit.

Back in Chapter 8, “Using Wildcard Filtering,” I mentioned the need to trim data in order to remove any trailing spaces. This can be done using the MySQL `RTrim()` function, as shown here:

► Input

```
SELECT Concat(RTrim(vend_name), ' (', RTrim(vend_country), ')')
FROM vendors
ORDER BY vend_name;
```

► Analysis

The `RTrim()` function trims all spaces from the right of a value. When you use `RTrim()`, the individual columns are all trimmed properly.

Note

The Trim() Functions In addition to `RTrim()` (which, as just seen, trims the right side of a string), MySQL supports the use of `LTrim()` (which trims the left side of a string) and `Trim()` (which trims both the right and left sides).

Tip

Functions Are Not Case-Sensitive As MySQL functions are not case-sensitive, you can concatenate by using `Concat()`, `concat()` and even `CONCAT()`. It comes down to personal preference, and you can feel free to use any style you prefer. But please be consistent: Whatever style you opt for, stick with it. Doing so will make your code much easier to read and maintain in the future.

Using Aliases

The `SELECT` statement that is used to concatenate the address field works well, as you can see in the previous output. But what is the name of the new calculated column? Well, the truth is, it has no name; it is simply a value. Although this can be fine if you are just looking at the results in a SQL query tool, an unnamed column cannot be used within a client application because the client has no way to refer to that column.

To solve this problem, SQL supports column aliases. An *alias* is an alternative name for a field or value. You assign aliases by using the `AS` keyword. Take a look at the following `SELECT` statement:

► Input

```
SELECT Concat(RTrim(vend_name), ' (', RTrim(vend_country), ')')
       AS vend_title
FROM vendors
ORDER BY vend_name;
```

► Output

```
+-----+
| vend_title |
+-----+
| ACME (USA) |
| Anvils R Us (USA) |
| Furball Inc. (USA) |
| Jet Set (England) |
| Jouets Et Ours (France) |
| LT Supplies (USA) |
+-----+
```

► Analysis

This `SELECT` statement is the same as the one used in the previous code snippet, except that here the calculated field is followed by the text `AS vend_title`. This instructs SQL to create a calculated field named `vend_title` that contains the results of the specified calculation. As you can see in the output, the results are the same as before, but the column is now named `vend_title`, and any client application can refer to this column by name, just as it would to any actual table column.

Tip

Other Uses for Aliases Aliases have other uses, too. Some common uses include renaming a column if the table column name contains illegal characters (for example, spaces) and expanding a column name if the original name is either ambiguous or easily misread.

Note

Derived Columns *Aliases* are also sometimes referred to as *derived columns*. These two terms mean the same thing.

Performing Mathematical Calculations

Another frequent use for calculated fields is performing mathematical calculations on retrieved data. Let's take a look at an example. The `orders` table contains all orders received, and the `orderitems` table contains the individual items within each order. The following SQL statement retrieves all the items in order number 20005:

► Input

```
SELECT prod_id, quantity, item_price
FROM orderitems
WHERE order_num = 20005;
```

► Output

prod_id	quantity	item_price
ANV01	10	5.99
ANV02	3	9.99
TNT2	5	10.00
FB	1	10.00

The `item_price` column contains the per-unit price for each item in an order. To expand the item price (item price multiplied by quantity ordered), you simply use the following:

► Input

```
SELECT prod_id,  
       quantity,  
       item_price,  
       quantity*item_price AS expanded_price  
FROM orderitems  
WHERE order_num = 20005;
```

► Output

prod_id	quantity	item_price	expanded_price
ANV01	10	5.99	59.90
ANV02	3	9.99	29.97
TNT2	5	10.00	50.00
FB	1	10.00	10.00

► Analysis

The `expanded_price` column shown in the previous output is a calculated field; the calculation is simply `quantity*item_price`. Once this calculated column is created, the client application can use it just as it would any other column.

MySQL supports the basic mathematical operators listed in Table 10.1. In addition, you can use parentheses to establish the order of precedence. Refer to Chapter 7, “Advanced Data Filtering,” for an explanation of precedence.

Table 10.1 MySQL Mathematical Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division

Tip

How to Test Calculations `SELECT` provides a great way to test and experiment with functions and calculations. Although `SELECT` is usually used to retrieve data from a table, the `FROM` clause may be omitted to simply access and work with expressions. For example, `SELECT 3 * 2;` would return 6, `SELECT Trim(' abc ');` would return `abc`, and `SELECT Now();` uses the `Now()` function to return the current date and time. You get the idea, and you can learn more by using `SELECT` to experiment.

Summary

In this chapter, you learned what calculated fields are and how to create them. You saw examples demonstrating the use of calculated fields for both string concatenation and mathematical operations. In addition, you learned how to create and use aliases so your application can refer to calculated fields.

Challenges

1. A common use for aliases is to rename table column fields in retrieved results (perhaps to match specific reporting or client needs). Write a SQL statement that retrieves `vend_id`, `vend_name`, `vend_address`, and `vend_city` from `Vendors` and rename `vend_name` to `vname`, `vend_city` to `vcity`, and `vend_address` to `vaddress`. Sort the results by vendor name; for this you can use either the original name or the new name.
2. Our example store is running a sale, and all products are 10% off. Write a SQL statement that returns `prod_id`, `prod_price`, and `sale_price` from the `Products` table. `sale_price` is a calculated field that contains the sale price. Here's a hint: You can multiply by 0.9 to get 90% of the original value (and thus the 10% off price).

This page intentionally left blank